Multi-Source Candidate Data Transformer

Abstract:

It takes messy candidate information from multiple places a recruiter's CSV spreadsheet and resume files (PDF/DOCX) and combines them into one clean, organized profile per candidate. It also tracks where each piece of information came from and how confident it is in each value.

Pipeline

Stage	Module	What it does (simple)
Detect	detect.py	Looks at each file and says: is this a CSV or a resume
Parse	parsers/	Reads the file and pulls out: name, email, phone, skills
Normalize	normalize.py	Cleans up data e.g. turns (415) 555-0101 into +14155550101
Merge	merge.py	If two files have the same email, combines them into 1 profile
Confidence	confidence.py	Gives each profile a score like 0.97 how sure are we
Project	project.py	Reshapes output based on a config file no code changes needed
Validate	validate.py	Checks the output looks correct before saving

How to Run

Step 1 - Install dependencies

```
pip install -r requirements.txt
```

Step 2 - Run with default output

```
python cli.py --input-dir sample_inputs --out output.json
```

You will see: Wrote 2 candidate profile(s) to output.json

Then open output.json to see the merged, normalized candidate profiles.

Step 3 - Run with custom config

```
python cli.py --input-dir sample_inputs --config schema/example_config.json --out output_custom.json
```

This reshapes the output - only the fields you want, renamed however you like.

Step 4 — Run the tests

```
python -m pytest tests/ -v
```

You should see 9 passed - all tests green.

What is the config file for?

Say you only want name, email, and skills in the output instead of everything - you write that in the config file and the pipeline reshapes the output without any code changes.

Config options:

- fields - which fields to include, what to rename them, how to normalize them
- include_confidence - show or hide the confidence score
- include_provenance - show or hide where each value came from
- on_missing - what to do if a field is empty: "null", "omit", or "error"

What does each file do?

File	What it does
cli.py	The main file you run ties everything together
detect.py	Figures out if a file is CSV or resume
parsers/csv_parser.py	Reads the recruiter spreadsheet
parsers/resume_parser.py	Reads PDF/DOCX resumes
normalize.py	Cleans up phones, dates, and skill names
merge.py	Combines records from multiple sources into one profile
confidence.py	Calculates the confidence score (0.0 to 1.0)
project.py	Reshapes output based on the config file
validate.py	Checks the output is correct before saving
schema/default_schema.json	The expected structure of the output
schema/example_config.json	Example custom config file
sample_inputs/	Test files a recruiter CSV and a resume DOCX
tests/test_pipeline.py	9 automated tests

Known Limitations (Deliberate Descopes)

- Only CSV and Resume sources are implemented (satisfies: at least 1 structured + 1 unstructured)
- Skill extraction uses keyword matching - won't catch skills not in the vocabulary list
- Date extraction from resume free-text is not implemented - returns null rather than guessing
- If two people have the same name and no email, they may wrongly merge - email is the primary match key
- No web UI - command line only (assignment says CLI is completely fine)

Design Decisions Worth Noting

- **Clean engine/projection separation** - canonical record is always fully built first; config only reshapes the output
- **Honest confidence formula** - deterministic weighted sum (email +0.3, phone +0.2, multi-source +0.2)
- **Provenance on every field** - you can trace exactly where each value came from and what method extracted it
- **Graceful degradation** - corrupt PDF Skip and warn. Bad phone? Return null, never invent a value
- **Caught and fixed a real bug** - 'go' was matching inside 'Django' via substring; fixed with word-boundary regex and added a regression test